

Learning goals

- Compute small DFTs by hand and use the DFT properties (periodicity, symmetry for real signals, Parseval).
- Distinguish frequency resolution $\Delta f = f_s/N$ (record length) from the bin spacing obtained by zero padding.
- Explain leakage and choose a window by main-lobe width, side-lobe level, scalloping loss and equivalent noise bandwidth.
- Implement an iterative radix-2 FFT and verify its $\mathcal{O}(N \log N)$ complexity against the $\mathcal{O}(N^2)$ direct DFT.
- Compute a crank-angle-triggered knock spectrum on the ESP32 and judge the memory and operation budget of an 8-bit Uno.

Background

The N -point DFT of $x[n]$, $n = 0, \dots, N-1$, and its inverse are

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}, \quad W_N = e^{-j2\pi/N}.$$

$X[k]$ samples the DTFT of the *finite record* at $f_k = k f_s/N$. Multiplying by a window $w[n]$ before the DFT convolves the spectrum with the window transform: a sinusoid between two bins leaks into all bins (*leakage*). The radix-2 decimation-in-time FFT splits an N -point DFT into two $N/2$ -point DFTs of the even and odd samples,

$$X[k] = G[k] + W_N^k H[k], \quad X[k + N/2] = G[k] - W_N^k H[k],$$

recursively, giving $\log_2 N$ stages of $N/2$ *butterflies*.

The FFT written on this sheet is used again on Sheets 11 and 12. Its interface is fixed (file `fft.h`):

```
1 void fft(double *re, double *im, int n, int inverse);
2 /* in place, n = power of two; inverse = 1 includes the factor 1/n */
```

Exercise 8.1 DFT properties**PAPER**

- Write down the 4×4 DFT matrix and compute the DFT of $x = \{1, 2, 0, -1\}$. Check Parseval's relation $\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$.
- Compute the 8-point DFT of $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ in closed form (geometric series) and give $|X[k]|$. Which symmetry do you observe and why must it hold?
- Resolution.** The knock measurement window extends from 10° to 70° after TDC, $f_s = 40$ kHz. How many samples does it contain at 3000 rpm and at 6000 rpm, and what is the frequency resolution Δf ? Can a knock component at 6.5 kHz be separated from the valve impact at 8 kHz (with a Hann window)? Compare Δf with the natural line width $B = 1/(\pi\tau)$ of the knock burst (Sheet 7).
- The window of c) at 3000 rpm is zero padded to $N = 256$. What changes in the spectrum, what does not? What *does* improve the resolution?

- e) **Windows.** Give main-lobe width (null to null, in bins) and highest side lobe for the rectangular, Hann, Hamming and Blackman windows. The knock component at 6.5 kHz is 60 dB stronger than a valve component at 8 kHz (9.6 bins away for $N = 256$). Which windows can reveal the weak component?
- f) **Complexity.** Count complex multiplications of the direct DFT and of the radix-2 FFT for $N = 256$. At 6000 rpm, how many 256-point FFTs per second does an angle-synchronous knock detection for a 4-cylinder engine need, and how many real multiplications per second is that?

Solution

a) $W_4 = -j$:

$$\begin{pmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 - 3j \\ 0 \\ 1 + 3j \end{pmatrix}.$$

Parseval: $1 + 4 + 0 + 1 = 6$ and $(4 + 10 + 0 + 10)/4 = 6$.

b) $X[k] = \sum_{n=0}^3 W_8^{kn} = \frac{1-W_8^{4k}}{1-W_8^k} = e^{-j3\pi k/8} \frac{\sin(\pi k/2)}{\sin(\pi k/8)}$ ($k \neq 0$), $X[0] = 4$. $X = \{4, 1 - 2.414j, 0, 1 - 0.414j, 0, 1 + 0.414j, 0, 1 + 2.414j\}$, $|X| = \{4, 2.613, 0, 1.082, 0, 1.082, 0, 2.613\}$ (confirmed by `fft_test`). For real x : $X[N - k] = X^*[k]$ (conjugate symmetry), $|X|$ is even around $N/2$; $X[0]$ and $X[N/2]$ are real. Only $N/2 + 1$ bins carry information.

c) At 3000 rpm: $18\,000^\circ/\text{s}$, $60^\circ \hat{=} 3.33\text{ ms}$, $N = 133$ samples, $\Delta f = 1/T_w = 300\text{ Hz}$. At 6000 rpm: 1.67 ms , $N = 67$, $\Delta f = 600\text{ Hz}$. The distance 1.5 kHz is 5 bins (3000 rpm) resp. 2.5 bins (6000 rpm); with the Hann main lobe (half width 2 bins) both are separable at 3000 rpm, only marginally at 6000 rpm. The natural line width of the burst is 398 Hz, i.e. at 3000 rpm the window already resolves the physical line; at high speed the window, not the physics, limits the resolution.

d) Zero padding to 256 samples evaluates the same DTFT of the 133-sample record on a finer grid (156 Hz spacing): peaks are located more accurately and look smoother, but two lines closer than $\approx 2\Delta f$ remain unresolved and the main lobe keeps its width. Only a longer record (more crank angle, lower speed) or a parametric method (Sheet 11) improves the resolution.

e)

window	main lobe (null-null)	highest side lobe
rectangular	2 bins	-13.3 dB
Hann	4 bins	-31.5 dB
Hamming	4 bins	-42.7 dB
Blackman	6 bins	-58.1 dB

At 9.6 bins distance the leakage of the strong line must stay below $\approx -60\text{ dB}$. Rectangular ($\approx -31\text{ dB}$ there) and Hamming (side lobes nearly flat at $-43 \dots -50\text{ dB}$) fail; Hann (side lobes decay with 18 dB/octave) and Blackman succeed. Exercise 8.3 confirms this numerically.

f) DFT: $N^2 = 65536$ complex multiplications; FFT: $\frac{N}{2} \log_2 N = 1024$ butterflies with one complex multiplication each (64 times fewer; many are trivial, $W^0 = 1$). 6000 rpm: 50 cycles/s $\times$ 4 cylinders = 200 windows/s, i.e. $200 \cdot 1024 \cdot 4 \approx 8.2 \cdot 10^5$ real multiplications per second (plus $1.2 \cdot 10^6$ additions) – easy for an ESP32, impossible with a direct DFT on a small ECU ($5.2 \cdot 10^7$ mult/s).

Exercise 8.2 Own FFT**JS LINUX**

Files: `code/jslinux/sheet08/fft.h` (template of the FFT header)
`code/jslinux/sheet08/fft_test.c` (test program)

- a) Implement the direct DFT `dft()` in `fft_test.c`. Avoid N^2 calls of `sin/cos`: tabulate W_N^m , $m = 0..N - 1$, and use the index $m = (kn) \bmod N$.

- b) Implement `fft()` in `fft.h`: bit-reversal permutation, $\log_2 N$ butterfly stages, $1/N$ scaling for the inverse. Compute the twiddle factors of a stage by the recursion $W_L^{k+1} = W_L^k W_L$ (one $\sin/\cos$ pair per stage). The program compares FFT and DFT for random complex data, checks $\text{IFFT}(\text{FFT } x) = x$, Parseval, and conjugate symmetry. Also check your results of Exercise 8.1 a) and b).
- c) Implement `fit_exponent()` (least squares in log-log coordinates). Measure the runtime of DFT and FFT versus N and determine the exponents. Plot $t_{\text{FFT}}/(N \log_2 N)$ – is it constant? Explain deviations for small and for large N .
- d) How does the round-off error of the FFT grow with N ? Why does the twiddle recursion increase it slightly, and how could one avoid that?

Solution

b) The reference implementation (`code/solutions/jslinux/sheet08/fft.h`):

```

1 static void fft(double *re, double *im, int n, int inverse)
2 {
3     int i, j, k, L, half, bit;
4     double sgn = inverse ? 1.0 : -1.0;
5
6     if (!fft_ispow2(n) || n < 2) return;
7
8     /* 1) bit-reversal permutation */
9     for (i = 1, j = 0; i < n; i++) {
10         bit = n >> 1;
11         while (j & bit) { j ^= bit; bit >>= 1; }
12         j |= bit;
13         if (i < j) {
14             double t;
15             t = re[i]; re[i] = re[j]; re[j] = t;
16             t = im[i]; im[i] = im[j]; im[j] = t;
17         }
18     }
19
20     /* 2) butterflies, stage by stage: L = length of the sub-DFTs */
21     for (L = 2; L <= n; L <<= 1) {
22         double ang = sgn * 2.0 * M_PI / L;
23         double cr = cos(ang), ci = sin(ang); /* W_L = e^{-+j 2 pi / L} */
24         half = L >> 1;
25         for (i = 0; i < n; i += L) { /* every sub-DFT block */
26             double wr = 1.0, wi = 0.0, t; /* W_L^0 */
27             for (k = 0; k < half; k++) {
28                 int a = i + k, b = a + half;
29                 double tr = wr * re[b] - wi * im[b]; /* W b */
30                 double ti = wr * im[b] + wi * re[b];
31                 re[b] = re[a] - tr; im[b] = im[a] - ti;
32                 re[a] += tr; im[a] += ti;
33                 t = wr * cr - wi * ci; /* W_L^{k+1} = W_L^k W_L */
34                 wi = wr * ci + wi * cr;
35                 wr = t;
36             }
37         }
38     }
39
40     /* 3) scaling of the inverse transform */
41     if (inverse) {
42         double s = 1.0 / n;
43         for (i = 0; i < n; i++) { re[i] *= s; im[i] *= s; }
44     }
45 }

```

Output: `fft` reproduces $X = \{2, 1 - 3j, 0, 1 + 3j\}$ and the 8-point result of 8.1 b). Comparison

with the DFT (random complex input):

N	$\max X_{\text{FFT}} - X_{\text{DFT}} $	$\max \text{IFFT}(\text{FFT}x) - x $	Parseval rel. error
8	$9.9 \cdot 10^{-16}$	$2.5 \cdot 10^{-16}$	$2.0 \cdot 10^{-16}$
128	$2.2 \cdot 10^{-14}$	$2.8 \cdot 10^{-15}$	$1.0 \cdot 10^{-15}$
2048	$1.5 \cdot 10^{-12}$	$3.6 \cdot 10^{-14}$	$4.5 \cdot 10^{-15}$

Real input ($N = 64$): $\max |X[k] - X^*[N - k]| = 7.3 \cdot 10^{-15}$, $\text{Im } X[0] = \text{Im } X[N/2] = 0$.

c) Measured on the reference PC (gcc -O2; JSLinux is about 50 times slower):

N	DFT [ms]	FFT [ms]	speed-up	$t_{\text{FFT}}/(N \log_2 N)$ [ns]
16	0.0003	0.00016	2.0	2.46
128	0.0210	0.00132	15.9	1.47
1024	1.413	0.0149	94.6	1.46
2048	6.339	0.0307	206.5	1.36
8192	–	0.178	–	1.67
32768	–	0.863	–	1.76

Fitted exponents: DFT $t \propto N^{2.02}$, FFT $t \propto N^{1.16}$ (a power-law fit to $N \log_2 N$ over $N = 64 \dots 32768$ gives an exponent slightly above 1, since $d \ln(N \log N)/d \ln N = 1 + 1/\ln N$). $t/(N \log_2 N)$ is almost constant (1.4–1.8 ns): for small N the fixed overhead (call, bit reversal, **sin/cos** per stage) dominates; for $N \geq 8192$ the data (16 B per complex sample, 128 kB) no longer fit into the L1/L2 cache and memory access becomes the bottleneck. Absolute times differ between machines, the exponents do not.

d) The error grows only slowly (roughly $\propto \log N$ per output for an ideal FFT, compared with $\propto N$ for the naive DFT summation). With the twiddle recursion each W_L^k accumulates k rounding errors, so the error rises to $1.5 \cdot 10^{-12}$ at $N = 2048$ (with exact **sin/cos** per k : $3.3 \cdot 10^{-13}$, measured with a first version of the code). Remedies: a precomputed twiddle table (best for repeated FFTs of the same size, as on the ESP32), or re-computing the twiddles exactly every few steps. In single precision (AVR, Exercise 8.4) the recursion error $\approx k \cdot 6 \cdot 10^{-8}$ is still negligible for $N \leq 128$ (inverse round trip error $5.5 \cdot 10^{-6}$).

Exercise 8.3 Windows, leakage and the knock spectrum

JSLINUX

Program `code/jslinux/sheet08/windows.c` uses your `fft.h`. Spectra are scaled as amplitude spectra, $A_k = 2|X[k]|/\sum w[n]$, so that a bin-centred sinusoid of amplitude A appears at $20 \log_{10} A$ dB.

- Implement the periodic Hann, Hamming and Blackman windows. The program computes coherent gain $\sum w/N$, equivalent noise bandwidth $\text{ENBW} = N \sum w^2/(\sum w)^2$, scalloping loss (tone exactly between two bins) and the highest side lobe. Compare with 8.1 e).
- Leakage: $x[n] = \sin(2\pi 6500 n/f_s) + 10^{-3} \sin(2\pi 8000 n/f_s)$, $N = 256$, $f_s = 40$ kHz. Which windows reveal the weak component? Explain the peak levels with the scalloping loss.
- Zero padding: two equally strong tones at 6500 Hz and 6900 Hz, records of $N = 64, 128, 256$ samples, each zero padded to 4096 points (Hann). Implement the parabolic interpolation of the peak position $\delta = \frac{1}{2} \frac{A_{k-1} - A_{k+1}}{A_{k-1} - 2A_k + A_{k+1}}$ (dB values). For which record length are the two tones resolved?
- Knock spectrum: record $N = 256$ samples of `es_knock()` at 40 kHz starting at the TDC of a knocking cylinder (3000 rpm, `knock_prob = 1`), i.e. 0° – 115° ATDC, and the window 10° – 68° ATDC (128 samples). Identify the peaks. Why is the knock peak higher in the shorter window although it contains less signal energy? Where is the valve impact (8 kHz, 90° ATDC)?

Solution

a) Program output ($N = 256$):

window	coherent gain	ENBW [bins]	scalloping [dB]	highest side lobe [dB]
rectangular	1.000	1.000	−3.89	−13.3
Hann	0.500	1.500	−1.42	−31.5
Hamming	0.540	1.363	−1.75	−42.7
Blackman	0.420	1.727	−1.10	−58.1

Low side lobes are paid for by a wider main lobe and a larger ENBW (more noise per bin, i.e. lower SNR of a single bin).

b) $\Delta f = 156.25$ Hz; 6500 Hz lies at bin 41.6, 8000 Hz at bin 51.2. Levels in dB:

window	peak	bin of 8 kHz	at 9.5 kHz	at 15 kHz
rectangular	−2.47	−31.0	−38.2	−50.7
Hann	−0.91	−58.8	−87.6	−114.4
Hamming	−1.12	−48.4	−55.0	−67.3
Blackman	−0.70	−59.7	−95.1	−121.8

The weak tone (−60 dB) is visible only with Hann and Blackman (the bins around 9.5 kHz are far lower, so the −59 dB at 8 kHz is the tone, not leakage). Rectangular and Hamming bury it under leakage of the strong tone. The peak levels are below 0 dB because the tone is 0.4 bins off the bin centre (scallop; maximum at 0.5 bins as in a)).

c) Hann, zero padded to 4096: $N = 64$ ($\Delta f = 625$ Hz): two maxima at 6112 Hz and 7288 Hz – *wrong*: they are the merged main lobe with its interference pattern, not the two lines. $N = 128$ ($\Delta f = 312.5$ Hz): a single peak at 6700 Hz (unresolved). $N = 256$ ($\Delta f = 156$ Hz, separation 2.56 bins): 6504.3 Hz and 6895.7 Hz, both 0.2 dB – resolved and accurately located. Zero padding gave a smooth curve in all cases but never created resolution. Key code:

```

1 static void print_peaks(const double *db, int nb, int nfft, double rel)
2 {
3     int k;
4     double thr = db[0];
5     for (k = 1; k < nb; k++) if (db[k] > thr) thr = db[k];
6     thr -= rel; /* relative threshold */
7     for (k = 2; k < nb - 2; k++) {
8         if (db[k] > thr && db[k] >= db[k - 1] && db[k] > db[k + 1] &&
9             db[k] >= db[k - 2] && db[k] > db[k + 2]) {
10             double a = db[k - 1], b = db[k], c = db[k + 1];
11             double d = 0.5 * (a - c) / (a - 2 * b + c); /* offset in bins */
12             printf("    peak at %8.1f Hz (bin %3d%+.2f) %6.1f dB\n",
13                 (k + d) * FS / nfft, k, d, b - 0.25 * (a - c) * d);
14         }
15     }
16 }
17
18 int main(void)
19 {

```

d) Seed 7, first knocking cylinder (cyl. 1, onset 15.8° ATDC), peaks within 15 dB of the maximum (Hann): 0°–115°, $N = 256$: 6508 Hz/−19.6 dB, 10 563 Hz/−24.4 dB (and a noise peak at 7349 Hz/−32.3 dB); 10°–68°, $N = 128$: 6513 Hz/−12.9 dB, 10 479 Hz/−17.8 dB. Second cylinder (cyl. 3, onset 19.3°): 6489 Hz/10 491 Hz resp. 6497 Hz/10 520 Hz. The amplitude scaling divides by $\sum w \propto N$: a burst that lasts only $\approx 3\tau = 2.4$ ms appears with its *average* amplitude over the window, so a window matched to the burst gives a higher level (and a better SNR, since the noise outside the burst is excluded). The valve burst starts at 90° ATDC = 5.0 ms, i.e. in the last 22 % of the long window, where the Hann window is small: only a weak 8 kHz contribution (about −34 dB) remains; the short window excludes it completely – another reason for the 10°–70° measurement window.

Exercise 8.4 FFT on the microcontroller**WOKWI**

Project `code/wokwi/sheet08_knockfft/` (ESP32 + `engine-sim` chip): KNOCK → D34 (ADC1), TDC → D18, KNK (ground truth) → D19. Add your `fft.h` of Exercise 8.2 to the project.

- The sketch waits for the rising edge of the TDC pulse and then records $N = 256$ samples at $f_s = 40$ kHz. Which crank-angle range does the record cover at 1500, 3000 and 6000 rpm? Why is the analysis synchronised to the crank angle and not to time?
- Implement the capture with a deadline loop (`micros()`, period 25 μ s); count missed deadlines in `late` and record whether KNK was high during the window.
- Implement the Hann window, the search for the largest bin above 3 kHz and the parabolic peak interpolation. Why must the search exclude the low-frequency range? Set the knock probability to 50% and collect about 20 windows: compare peak frequency, peak level and the knock-band energy `e_knock` (6–7 kHz) for windows with and without knock (`knk`). Plot one spectrum in JSLinux from the CSV output.
- Uno.** Open `code/wokwi/sheet08_unoffft/` (synthetic test signal, same `fft.h`; on the AVR `double` is a 32-bit float). Compile for $N = 64, 128, 256$ and read the memory report. What is the largest feasible N and why? Add the operation count (butterflies, real multiplications and additions). Estimate the execution time from the operation count, assuming roughly 100–150 clock cycles per soft-float operation at 16 MHz, and compare with the time between two TDCs at 3000 rpm. Suggest two ways to fit a 256-point spectrum into the Uno.

Hint: Wokwi is not cycle-accurate: `micros()` differences (`tcap`, `tfift`) are simulated times and only indicative. The Serial output of one window (130 lines) takes about 0.25 s at 115200 baud; TDC events during printing are simply skipped.

Solution

a) $256/f_s = 6.4$ ms, i.e. $6 \cdot n \cdot 6.4$ ms degrees: 57.6° at 1500 rpm, 115.2° at 3000 rpm, 230.4° at 6000 rpm (longer than the 180° between two TDCs – the next combustion leaks in; a real system uses an angle-dependent N or f_s). Knock can only occur in a known angle range after TDC; triggering on TDC makes the windows of all combustions comparable and excludes disturbances at known angles (valve closing).

b)/c) Key part of `code/solutions/wokwi/sheet08_knockfft/sketch.ino`:

```

1  /* 1) wait for the rising edge of the TDC pulse */
2  while (digitalRead(PIN_TDC) == HIGH) { }
3  while (digitalRead(PIN_TDC) == LOW) { }
4
5  /* 2) capture N samples with a deadline loop */
6  t0 = tn = micros();
7  for (n = 0; n < N; n++) {
8      if ((long)(micros() - tn) > 0) late++;           /* deadline missed */
9      while ((long)(micros() - tn) < 0) { }
10     tn += TS_US;
11     raw[n] = analogRead(PIN_KNOCK);
12     if (digitalRead(PIN_KNK)) knk = 1;
13 }
14 tcap = micros() - t0;
15
16 /* 3) sensor volts, remove mean, Hann window */
17 for (n = 0; n < N; n++) mean += raw[n];
18 mean /= N;
19 for (n = 0; n < N; n++) {
20     double w = 0.5 - 0.5 * cos(2.0 * M_PI * n / N);
21     re[n] = (raw[n] - mean) * (VREF / 4095.0) / OFFSET_GAIN * w;
22     im[n] = 0.0;
23     sw += w;

```

```

24     }
25
26     /* 4) FFT and amplitude spectrum in dB (sinusoid of amplitude A -> 20 log A) */
27     t0 = micros();
28     fft(re, im, N, 0);
29     tfft = micros() - t0;
30     kp = (int)(F_MIN * N / FS);
31     for (k = 0; k <= N / 2; k++) {
32         double a = 2.0 * sqrt(re[k] * re[k] + im[k] * im[k]) / sw;
33         db[k] = 20.0 * log10(a + 1e-9);
34         if (k * FS / N >= F_MIN && db[k] > db[kp]) kp = k;
35         /* energy in the knock band 6.0 .. 7.0 kHz */
36         if (k * FS / N >= 6000.0 && k * FS / N <= 7000.0) e_knock += a * a;
37     }
38
39     /* 5) peak frequency with parabolic interpolation */
40     double d = 0.0;
41     if (kp < N / 2) {
42         double a = db[kp - 1], b = db[kp], c = db[kp + 1];
43         d = 0.5 * (a - c) / (a - 2.0 * b + c);
44     }

```

The global maximum is the combustion vibration (half sine over 60° , 0.2 V): after the mean removal it still produces a peak near 200 Hz of about -19 dB, as high as the knock peak; hence the search starts at 3 kHz. Expected results (PC emulation of the sketch with the same engine model, 50 % knock probability, 12-bit quantisation; Wokwi gives other random values but the same picture):

	peak frequency	peak level	e_knock
knocking (knk=1, 8 windows)	6470–6531 Hz	-23.9 to -15.5 dB	-19.7 to -11.0 dB
no knock (8 windows)	random, 7.8–19.8 kHz	-36 to -33 dB	-40.1 to -32.8 dB

The knock band energy separates the two classes by more than 10 dB – the idea of the detector of Sheet 12. The second mode (10.5 kHz) appears about 4–10 dB below the first. `late` should be 0; if not, the simulated `analogRead` is slower than $25\ \mu\text{s}$ (the ESP32 ADC needs about 10–20 μs in reality; a real application uses the I2S/DMA ADC mode).

d) arduino-cli memory report (global variables of 2048 B):

N	arrays $2 \cdot 4N$	global RAM	result
64	512 B	716 B (34 %)	OK
128	1024 B	1228 B (59 %)	OK, 820 B left for stack
256	2048 B	2252 B (109 %)	data section exceeds available space

Largest $N = 128$ (about 200 B are taken by the Serial buffers and the core, the stack needs the rest). Operation count for $N = 128$: $\frac{N}{2} \log_2 N = 448$ butterflies, $4 \cdot 448 = 1792$ real multiplications, $6 \cdot 448 = 2688$ additions (plus the twiddle recursion, 4 mult. per butterfly group). With ≈ 120 cycles per float operation: $\approx 4500 \cdot 120 \approx 5.4 \cdot 10^5$ cycles ≈ 34 ms – more than three times the 10 ms between two TDCs at 3000 rpm. Expected strong bins of the test signal ($\Delta f = 312.5$ Hz): bin 21 (6562.5 Hz, 0.92) and its neighbours for 6.5 kHz, bins 33/34 (0.25/0.38) for 10.5 kHz (leakage, no window). Ways out: (1) a real-input FFT (an N -point real FFT via an $N/2$ -point complex FFT) halves the memory; (2) 16-bit fixed point (Q15) instead of float: 4 B per complex sample, $N = 256$ needs 1 kB and integer multiplications are much faster than soft-float; (3) do not compute a full spectrum at all – a Goertzel filter or a band-pass (Sheets 9/10) computes the knock-band energy with a few bytes of state.

Deliverables

DFT computations of 8.1; your `fft.h` (it is reused on Sheets 11 and 12!) with the error and runtime tables of 8.2; the window tables and the resolution experiment of 8.3; for 8.4 a CSV spectrum of one knocking and one non-knocking window with your knock/no-knock statistics, and the Uno memory table with the operation-count estimate.